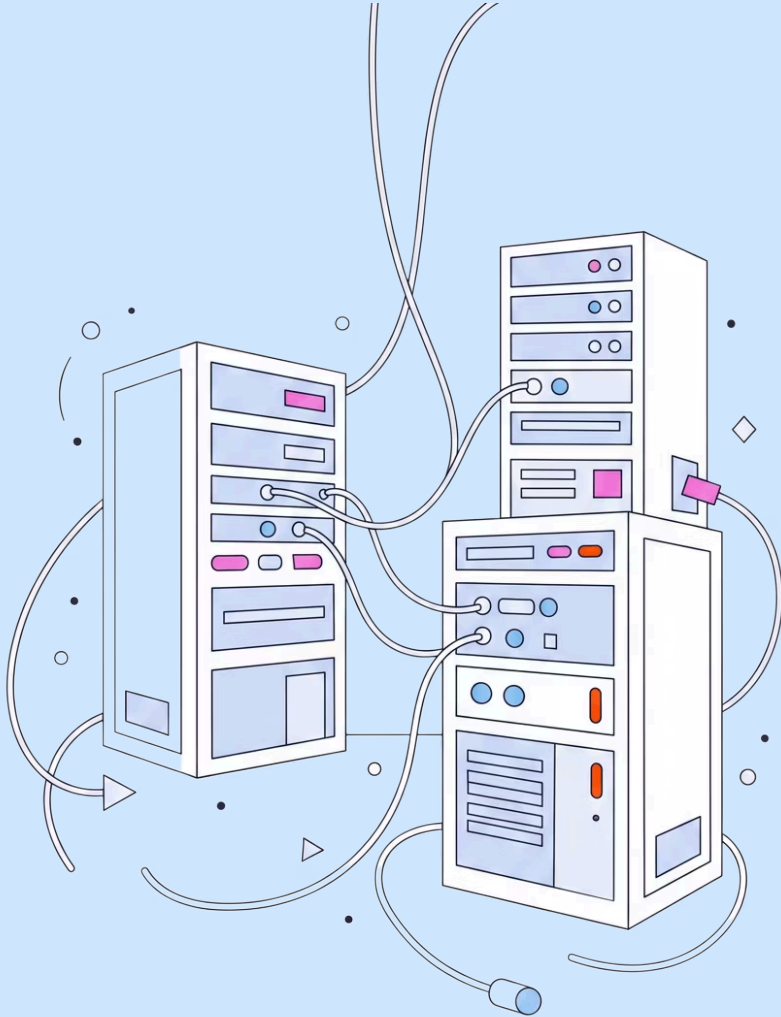




La VPS: aprovisionar, entrar y cerrar

Capítulo 2 · Unidad de estudio con fundamentos teóricos

VPS · SSH · SEGURIDAD · FIREWALL

Alcance del capítulo

Este capítulo cubre la puesta en marcha del servidor donde vivirá la aplicación. El énfasis no está en el aprovisionamiento —que en la práctica se reduce a unos pocos clics— sino en lo que ocurre inmediatamente después: **un servidor recién creado es, por defecto, un objetivo.**

01

Virtualización y VPS

Origen, diseño e ilusión controlada

02

SSH y claves

Arquitectura, autenticación y acceso seguro

03

Puertos y firewall

Sockets, netfilter y ufw

04

Docker y seguridad

El problema del firewall, riesgos y endurecimiento

❏ Al finalizar, cada grupo debe contar con un VPS operativo, accesible por claves SSH, con el firewall configurado y el panel de administración publicado bajo su propio dominio con certificado válido.



Por qué existe un servidor virtual: origen de la virtualización

Un VPS no es un tipo de computadora: es una **ilusión cuidadosamente construida**. El grupo recibe lo que parece una máquina completa —con su propio sistema operativo, su propia IP, su propio disco— cuando en realidad está compartiendo hierro con decenas de clientes desconocidos.

El problema es viejo y el punto de partida fue económico. En los años sesenta, una computadora central costaba una fortuna y se usaba de a un programa por vez. IBM atacó el problema con VM/370 en 1972: en lugar de repartir el tiempo, el programa de control creaba **varias máquinas virtuales completas**, cada una con la ilusión de tener el hardware para sí sola.

- La diferencia decisiva con el tiempo compartido: en la virtualización cada usuario tiene su propio sistema operativo y **no sabe que los demás existen**.

Las tres propiedades de Popek y Goldberg

En 1974, Gerald Popek y Robert Goldberg formalizaron el problema: enunciaron las tres propiedades que debe cumplir un sistema para llamarse virtualizador, y demostraron qué condición debe cumplir un juego de instrucciones para admitirlas.

Equivalencia

Los programas se comportan igual que sobre el hardware real. El huésped no puede distinguir si corre sobre una máquina virtual o sobre el hierro físico.

Control de recursos

El hipervisor tiene la última palabra sobre el hardware. Ningún programa huésped puede acceder a recursos sin que el hipervisor lo autorice.

Eficiencia

La mayoría de las instrucciones se ejecutan directamente sobre el procesador, sin emulación. El costo de la virtualización debe ser pequeño.

❏ La arquitectura x86 **no cumplía esa condición** originalmente. Por eso la virtualización tardó treinta años en llegar al mundo de los servidores comunes: VMware (1999), Xen (2003), y finalmente las extensiones VT-x de Intel y AMD-V (~2006).

Tipos de hipervisor y consecuencias prácticas

La industria distingue dos ubicaciones posibles para el **hipervisor**, el programa que crea y arbitra las máquinas virtuales.

Tipo	Dónde corre	Ejemplos	Uso típico
Tipo 1 (nativo)	Directamente sobre el hardware	KVM, Xen, VMware ESXi, Hyper-V	Proveedores de nube y VPS
Tipo 2 (alojado)	Como un programa dentro de un SO	VirtualBox, VMware Workstation	Escritorio, laboratorio

Aislamiento real, recursos compartidos

El SO del grupo no puede ver al de otro cliente: son espacios de memoria separados que el procesador hace cumplir por hardware. Pero el procesador físico, el disco y la placa de red sí son los mismos. De ahí el **vecino ruidoso**.

El estado es un archivo

Destruir y recrear un servidor es una operación de archivos, no de hardware. Por eso "cambiar el sistema operativo" tarda minutos, y la operación **es destructiva y no tiene deshacer**.

La responsabilidad se corta en un punto

El proveedor responde por el hipervisor, el hardware y la red. Todo lo que ocurre dentro del sistema operativo huésped **es del titular**. El proveedor, por diseño, no puede ver adentro.

Qué es un VPS y dónde se ubica entre las alternativas

Un **servidor privado virtual** es una porción aislada de un servidor físico, con su propio sistema operativo, sus propios recursos asignados y acceso administrativo completo. La tabla siguiente lo sitúa respecto de las alternativas, ordenadas por cuánto se administra y cuánto se delega.

Modalidad	Qué se administra	Qué se delega	Costo
Alojamiento compartido	Archivos y base de datos	Todo lo demás	Muy bajo
PaaS (Vercel, Railway)	La aplicación	SO, red, escalado	Bajo a medio
VPS	Todo el sistema operativo	Solo el hardware	Medio
Servidor dedicado	Todo, incluido el hardware	Nada	Alto

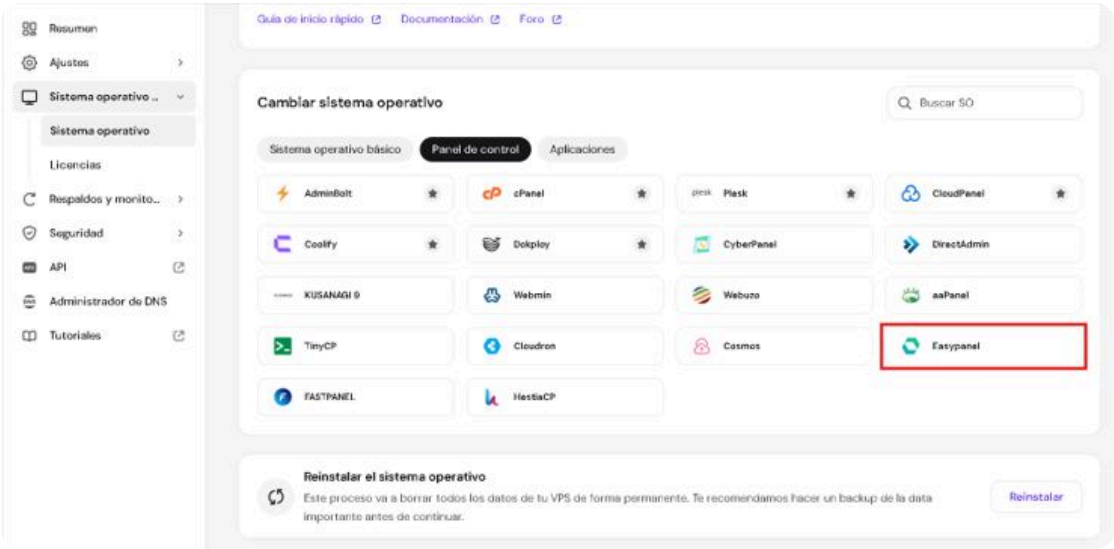
❏ La elección del VPS para este módulo es deliberada: un PaaS resolvería el despliegue en menos pasos, pero ocultaría la red, los puertos, el proxy inverso, los certificados y el aislamiento entre servicios. **Lo que no se observa no se aprende.**

2.4. Aprovisionamiento

Hostinger ofrece una plantilla de sistema operativo con Easypanel preinstalado sobre Ubuntu 24.04, que evita la instalación manual. Lo que la plantilla hace es escribir una imagen de disco preparada de antemano: por eso el proceso es tan rápido y por eso es irreversible.

Requisitos mínimos del práctico

Recurso	Mínimo	Motivo
Memoria RAM	2 GB	Requisito de Easypanel
Almacenamiento	20 GB	Imágenes de Docker
Sistema operativo	Ubuntu 24.04	Base de la plantilla
Puertos 80 y 443	Libres	Traefik toma control exclusivo



Procedimiento

1. Acceder a hPanel y seleccionar el servidor en la sección **VPS**.
2. Ingresar a la opción de cambio de sistema operativo.
3. Seleccionar la plantilla **Easypanel** y confirmar.
4. Aguardar la finalización del proceso.

2.5. Acceso remoto: el protocolo SSH

SSH (*Secure Shell*) es el protocolo mediante el cual se obtiene una consola remota del servidor. Su origen es una respuesta directa a un problema concreto: hasta mediados de los noventa, Telnet y rlogin transmitían **la contraseña en texto plano**. En 1995, Tatu Ylönen escribió la primera versión de SSH. La versión 2, la única en uso hoy, quedó normada en 2006 en las RFC 4251 a 4254.

Capa de transporte (RFC 4253)

Establece un canal cifrado e íntegro, y **autentica al servidor ante el cliente**, no al revés. El servidor presenta su clave de anfitrión y demuestra que la posee.

Capa de autenticación (RFC 4252)

Ahora sí, el cliente demuestra quién es. Es acá donde se elige entre contraseña y clave pública. La diferencia no es de comodidad sino de seguridad.

Capa de conexión (RFC 4254)

Multiplexa sobre el mismo canal la sesión interactiva, la copia de archivos y los túneles de puertos.

- ❏ **Trust on first use (TOFU):** la primera vez que se conecta a un servidor, el cliente no puede verificar la clave de anfitrión: pregunta, y si el usuario acepta, la guarda en `~/.ssh/known_hosts`. Si alguna vez aparece la advertencia de que la clave cambió, **no es un trámite**: o el servidor se reinstaló, o alguien se está interponiendo en el camino.

Contraseña vs. clave pública

La autenticación del cliente admite dos formas, y la diferencia entre ellas no es de comodidad sino de seguridad. La autenticación por clave es un **desafío criptográfico**: el servidor envía un dato aleatorio, el cliente lo firma con su clave privada, y el servidor verifica esa firma. La clave privada **nunca se transmite**.

	Contraseña	Clave pública
Qué viaja por la red	Un secreto reutilizable	Nada secreto
Resistencia a fuerza bruta	Baja	Prácticamente total
Qué pasa si se filtra el servidor	El atacante obtiene acceso	Nada: la clave privada nunca estuvo ahí
Cómo se revoca el acceso a una persona	Cambiando la contraseña de todos	Borrando una línea

Generación del par de claves

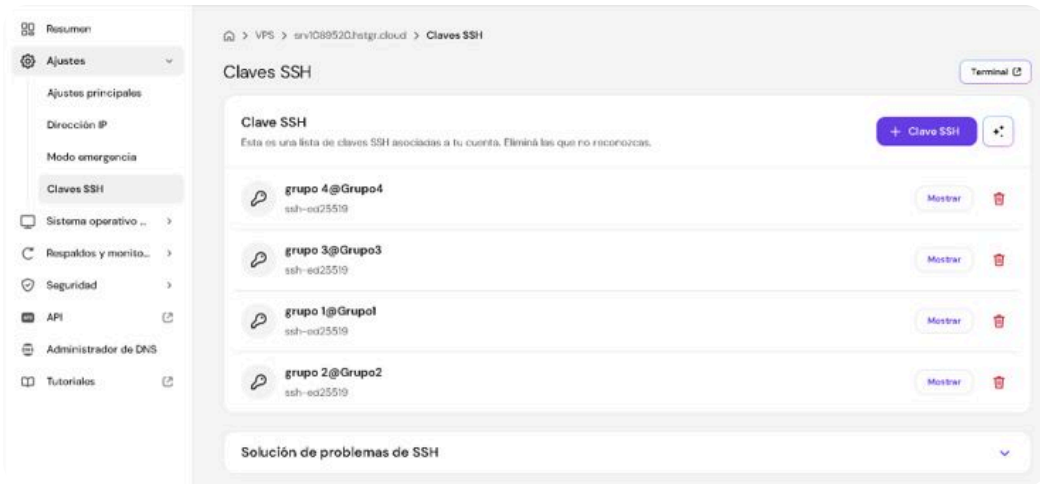
En el equipo del alumno, **no en el servidor**:
`ssh-keygen -t ed25519 -C "apellido-prog3"`
Genera `id_ed25519` (privada, nunca se comparte) e `id_ed25519.pub` (pública, se copia al servidor).

Instalación y conexión

En hPanel → **SSH Keys** se agrega el contenido de cada archivo `.pub`. Cada integrante instala su propia clave: cuatro accesos independientes, revocables por separado, sin contraseña compartida.
`ssh root@tudominio.com`

Instalación de claves y deshabilitación de contraseña

Claves del grupo en hPanel



Cuatro claves públicas independientes: un acceso por integrante, revocable por separado. Una contraseña compartida por cuatro personas es, a efectos de auditoría, una identidad sin dueño.

Deshabilitar el acceso por contraseña

Una vez verificado que las cuatro claves funcionan, editar `/etc/ssh/sshd_config`:

```
PasswordAuthentication no  
PermitRootLogin prohibit-password
```

Y recargar el servicio:

```
sudo systemctl reload ssh
```

`prohibit-password` no prohíbe entrar como root: prohíbe entrar como root **con contraseña**. El acceso por clave sigue permitido.

⚠ **Verificá que las claves funcionen ANTES de deshabilitar la contraseña.** Si te equivocás y cerrás la única sesión, hay que entrar por la consola de emergencia de Hostinger.

2.6. Puertos y sockets: qué significa "escuchar"

Una dirección IP identifica una máquina. Un **puerto** identifica un servicio dentro de esa máquina. Pero la formulación precisa es más útil: en la pila TCP/IP, una conexión establecida se identifica por una **tupla de cuatro elementos**: dirección de origen, puerto de origen, dirección de destino y puerto de destino.

El socket como tupla

Dos conexiones distintas al mismo servidor y al mismo puerto **no se confunden** porque difieren en el par de origen. Por eso un servidor web con un único puerto 443 puede atender diez mil conexiones simultáneas: no hay diez mil puertos, hay diez mil tuplas distintas.

Cuando un proceso se pone a **escuchar** en un puerto, le dice al sistema operativo: *entregame a mí toda conexión nueva dirigida a este número*. Dos procesos no pueden escuchar simultáneamente en la misma combinación de puerto e interfaz.

En qué interfaz escucha

Interfaz	Significado	Quién puede conectarse
127.0.0.1	Solo la máquina local	Únicamente procesos del mismo equipo
0.0.0.0	Todas las interfaces	Cualquiera que llegue por la red

Un servicio que escucha en 127.0.0.1 **es inalcanzable desde internet aunque el firewall esté apagado**: simplemente no hay nadie escuchando del lado de afuera. Es la forma más barata y más robusta de cerrar algo.

Rangos de puertos e inspección del estado

Los 65535 puertos no son todos iguales. La IANA los divide en tres rangos con procedimientos distintos, normados en la RFC 6335.

Rango	Nombre	Quién los asigna	Ejemplos
0 – 1023	Bien conocidos	IANA, por trámite formal	22 SSH · 53 DNS · 80 HTTP · 443 HTTPS
1024 – 49151	Registrados	IANA, por trámite simplificado	3000, 5432 PostgreSQL, 6379 Redis
49152 – 65535	Dinámicos o efímeros	Nadie: los toma el sistema operativo	El puerto de origen de tu navegador

En los sistemas Unix, **escuchar en un puerto por debajo de 1024 requiere privilegios de administrador**. Es la razón por la que los servidores de aplicaciones escuchan en 8000, 3000 o 5000 y hay un proxy inverso adelante ocupando el 80 y el 443.

```
sudo ss -tlnp
```

Bandera	Significado	
-t	Solo TCP	
-l	Solo sockets en escucha	Pide los sockets en estado LISTEN, los que esperan conexiones nuevas
-n	Números, sin resolver nombres	
-p	Mostrar el proceso responsable	

Figura 2.3 — Puertos en escucha en el servidor

```
https://ubuntu.com/engage/secure-kubernetes-at-the-edge
Expanded Security Maintenance for Applications is not enabled.
64 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

14 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

1 updates could not be installed automatically. For more details,
see /var/log/unattended-upgrades/unattended-upgrades.log

*** System restart required ***
Last login: Thu May 28 13:27:32 2026 from [REDACTED]
root@srv1089528:~# sudo ss -tlnp
State  Recv-Q  Send-Q  Local Address:Port  Peer Address:Port  Process
LISTEN 0        4096    0.0.0.0:27017       0.0.0.0:*           users:({"docker-proxy",pid=4738,fd=7})
LISTEN 0        511     127.0.0.1:18791    0.0.0.0:*           users:({"MainThread",pid=2310578,fd=27})
LISTEN 0        511     127.0.0.1:18789    0.0.0.0:*           users:({"MainThread",pid=2310578,fd=25})
LISTEN 0        4096    127.0.0.54:53      0.0.0.0:*           users:({"systemd-resolve",pid=2180501,fd=17})
LISTEN 0        4096    0.0.0.0:3000      0.0.0.0:*           users:({"docker-proxy",pid=4967,fd=7})
LISTEN 0        4096    0.0.0.0:443       0.0.0.0:*           users:({"docker-proxy",pid=4891,fd=7})
LISTEN 0        4096    0.0.0.0:22        0.0.0.0:*           users:({"sshd",pid=2023303,fd=3}, {"systemd",pid=1,fd=102})
LISTEN 0        4096    0.0.0.0:80        0.0.0.0:*           users:({"docker-proxy",pid=4873,fd=7})
```

En un servidor recién provisionado con la plantilla, la salida típica de `ss -tlnp` incluye cuatro procesos escuchando. Cada puerto abierto es **un proceso concreto que alguien decidió levantar**, no una propiedad mágica del servidor.

Puerto	Proceso	Función
22	sshd	Acceso administrativo
80	Traefik	HTTP y validación de certificados
443	Traefik	HTTPS
3000	Easypanel	Panel de administración

De estos cuatro, ¿cuáles necesita ver el mundo entero? La respuesta —80 y 443— es toda la clase de seguridad resumida.

2.7. El firewall: anatomía de netfilter

Un firewall decide qué paquetes entran y cuáles se descartan, antes de que lleguen al proceso que escucha. En Linux esa función no la cumple un programa sino un subsistema del propio núcleo, llamado **netfilter**, y las herramientas de línea de comandos —iptables, nft, ufw— no son el firewall: son formas de escribir reglas.

Netfilter define **puntos de enganche** (*hooks*) en lugares precisos del recorrido que un paquete hace dentro del núcleo. En cada punto se evalúan las reglas que allí haya registradas.

Punto de enganche	Cuándo se evalúa
PREROUTING	Apenas llega el paquete, antes de decidir a dónde va
INPUT	El paquete va a un proceso de esta misma máquina
FORWARD	El paquete atraviesa la máquina hacia otro destino
OUTPUT	El paquete lo generó un proceso local
POSTROUTING	Justo antes de salir por la placa de red

❏ La pieza decisiva está entre PREROUTING y los dos siguientes: **la decisión de enrutamiento**. Si el paquete es para esta máquina, sigue por INPUT. Si es para otra, sigue por FORWARD. Son dos caminos excluyentes: un paquete pasa por uno o por el otro, nunca por los dos.

Figura 2.4 — Recorrido de un paquete por netfilter

La decisión de enrutamiento separa dos caminos excluyentes: **ufw escribe sus reglas en INPUT**; el tráfico hacia los contenedores de Docker pasa por **FORWARD**. Reténgase este esquema, porque toda la sección 2.8 es una consecuencia suya.

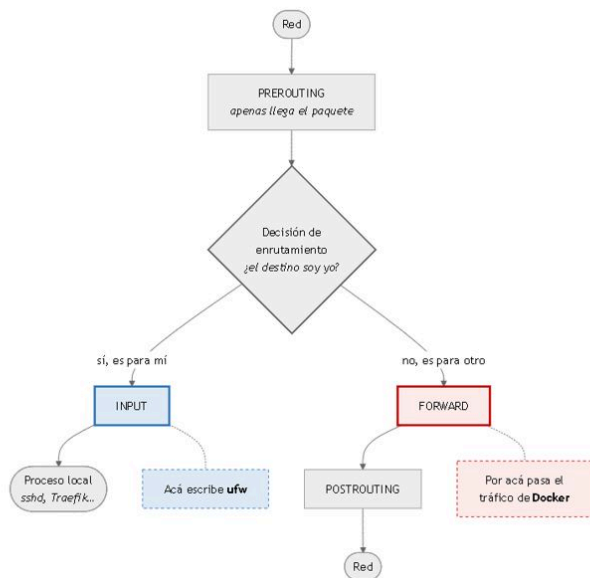


Tabla filter

Para aceptar o descartar paquetes. Aquí viven las reglas de ufw, en la cadena INPUT.

Tabla nat

Para reescribir direcciones. La traducción de direcciones que hace Docker vive aquí, convirtiendo el puerto del anfitrión en el puerto del contenedor.

Tabla mangle

Para modificar campos del paquete. Uso especializado, no se usa en este práctico.

2.7.2. Configuración del firewall con ufw

ufw (*Uncomplicated Firewall*) es una capa de conveniencia sobre netfilter: traduce órdenes legibles a reglas de netfilter. La configuración correcta parte de una **política restrictiva** —descartar todo lo que no esté explícitamente permitido— y abre solo lo necesario.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

Verificación:

```
sudo ufw status verbose
```

Puerto	Estado	Justificación
22	Abierto	Sin esto se pierde el acceso administrativo
80	Abierto	Redirección a HTTPS y validación de Let's Encrypt
443	Abierto	Todo el tráfico de la aplicación
Todo lo demás	Cerrado	No hay ningún motivo para exponerlo

⚠ El orden importa. Si corréis `ufw enable` **antes** de permitir el 22, el firewall corta tu propia sesión SSH en el acto. Permitir primero, habilitar después. Siempre.

2.8. El problema: Docker no respeta el firewall

Esta sección es probablemente la más importante del capítulo, y la que menos aparece en los tutoriales de despliegue. Ahora que está sobre la mesa el recorrido del paquete, el problema deja de ser un misterio y pasa a ser una consecuencia previsible.

Por qué ocurre

Un contenedor no comparte la pila de red del anfitrión: tiene su propia IP privada. Desde el punto de vista del núcleo, un contenedor es *otra máquina* alcanzable a través del anfitrión.

El recorrido del paquete

Docker reescribe la dirección de destino en PREROUTING (tabla nat). El núcleo concluye que el paquete **no es para él** y lo envía por FORWARD. **Nunca pasa por INPUT**. Y las reglas de ufw viven en INPUT.

La consecuencia

`sudo ufw deny 5432` — un contenedor levantado con `-p 5432:5432` queda **accesible desde internet igualmente**, y ufw status informa que el puerto está bloqueado. Las dos cosas son ciertas al mismo tiempo.

Lo que se cree	Lo que ocurre
ufw protege todos los puertos	ufw no ve el tráfico dirigido a puertos publicados por Docker
ufw status refleja el estado real	Refleja únicamente las reglas de ufw
Un puerto no listado está cerrado	Puede estar abierto por Docker

2.8.2. Verificación desde el exterior

La única comprobación confiable consiste en consultar los puertos desde otra máquina. La razón es epistemológica antes que técnica: cualquier herramienta que corra **dentro** del servidor está describiendo lo que ella cree; solo una prueba desde afuera atraviesa el mismo camino que atravesaría un atacante.

```
nmap -Pn tudominio.com
```

```
root@srv1089520:~# nmap -Pn 72.61.33.27
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-07-29 14:22 UTC
Nmap scan report for srv1089520.hstgr.cloud (72.61.33.27)
Host is up (0.000010s latency).
Not shown: 995 closed tcp ports (reset)
PORT      STATE      SERVICE
22/tcp    open      ssh
80/tcp    filtered  http
443/tcp   filtered  https

Nmap done: 1 IP address (1 host up) scanned in 1.27 seconds
root@srv1089520:~#
```

La bandera `-Pn` indica a nmap que no intente primero averiguar si el anfitrión está vivo mediante un ping, sino que escanee directamente. Muchos proveedores descartan los pings entrantes, y sin esa bandera nmap concluiría que el servidor no existe.

⚠ Leé esto dos veces: **ufw status te puede mentir**. No es un bug ni una mala configuración, es cómo funciona Docker. Un montón de bases de datos expuestas a internet en todo el mundo están así porque alguien corrió `ufw deny` y se quedó tranquilo.

2.8.3. Cómo se evita el problema

Existen tres estrategias, en orden de preferencia.

Estrategia	Cómo	Cuándo se usa
No publicar el puerto	No declarar mapeo de puertos en absoluto	Servicios internos: bases de datos, cachés
Publicar solo en local	-p 127.0.0.1:5432:5432	Acceso puntual mediante túnel SSH
Reglas en DOCKER-USER	Escribir reglas de iptables en esa cadena	Casos excepcionales

La primera estrategia es la que se aplica en este módulo

Un servicio que no publica puertos sigue siendo perfectamente accesible para los demás servicios del mismo proyecto, a través de la **red interna de Docker**. Ese mecanismo es el contenido central de la Clase 5.

La segunda estrategia: publicar en local

Al especificar 127.0.0.1 en el lado del anfitrión, la regla de traducción solo acepta paquetes que ya venían de la propia máquina. Es la forma correcta de conectarse a la base con una herramienta de escritorio: se abre un túnel SSH, y para el servidor la conexión es local.

❏ **La red interna no es una optimización, es la solución a este problema.** El Postgres de la Clase 5 no va a publicar el 5432 a ningún lado. La API le va a hablar por adentro. El puerto más seguro es el que no existe.

2.9. Riesgos concretos de la exposición innecesaria

No hace falta que nadie conozca la dirección del servidor. El espacio completo de direcciones IPv4 son unos 4.300 millones de direcciones, y escanearlo entero por un puerto determinado lleva minutos con herramientas de uso corriente. Existen además buscadores especializados —Shodan, Censys— que mantienen un índice permanente de qué servicio corre en cada dirección del mundo.

❏ **Un VPS recién creado empieza a recibir intentos de acceso dentro de la primera hora de existir.** Escanear internet entera no es una hazaña: es una tarea programada. Y como el resultado se indexa y se vende, el atacante ni siquiera necesita escanear.

Puerto	Servicio	Qué ocurre si se expone sin necesidad
22	SSH	Fuerza bruta continua. Con contraseña débil, compromiso total
3306	MySQL	Credenciales por defecto muy conocidas; lectura y borrado de datos
5432	PostgreSQL	Ídem
6379	Redis	Sin contraseña por defecto. Escritura arbitraria y ejecución remota
27017	MongoDB	Históricamente sin autenticación por defecto; miles de bases secuestradas
3000	Panel de admin	Contraseña en texto plano si no hay HTTPS
9000	Portainer y similares	Control total del motor de contenedores

Consecuencias que exceden al servidor

El daño de un servidor comprometido rara vez se limita a los datos que contiene.



Minado de criptomonedas

Consume el 100 % de los recursos y genera la factura. El servidor pasa a ser un recurso computacional ajeno.



Ataques de denegación de servicio

El servidor pasa a ser el atacante contra terceros, y la responsabilidad legal es del titular.



Envío de correo no deseado

La dirección IP queda en listas negras. El daño reputacional puede durar meses.



Movimiento lateral

Si desde ese servidor hay claves SSH hacia otros equipos, el compromiso se propaga. Un servidor comprometido con claves privadas adentro es **todo lo que esas claves abren**, comprometido.

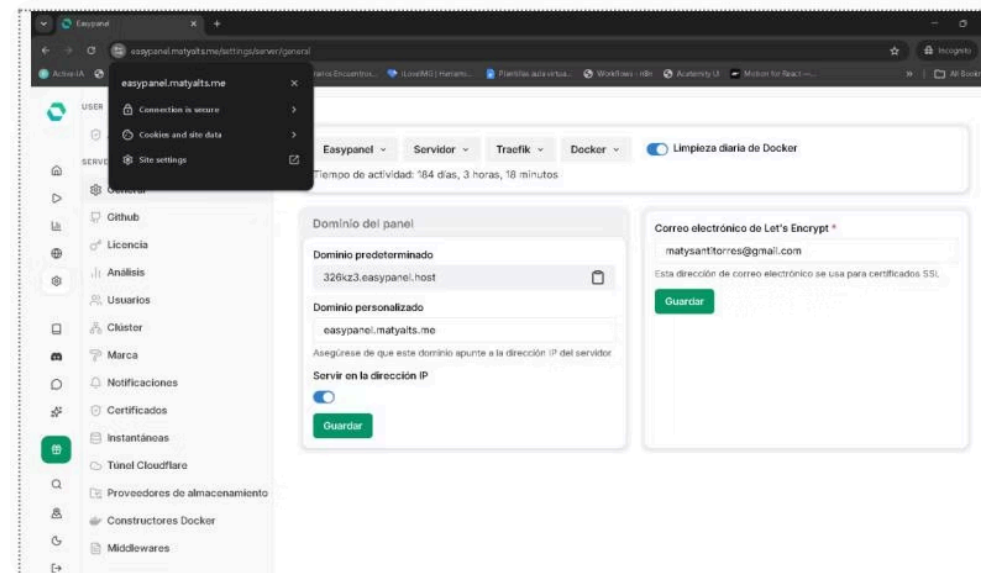
2.10. Publicación segura del panel de administración

Tras la instalación, el panel de Easypanel queda accesible en `http://IP_DEL_VPS:3000`. Esa configuración presenta dos problemas simultáneos: la contraseña del administrador viaja en texto plano, y el puerto está publicado por Docker con lo cual ufw no lo protege.

Procedimiento

1. Ingresar a **Settings** → **Domain** en Easypanel.
2. Configurar `easypanel.tudominio.com`.
3. Guardar y aguardar la emisión del certificado.
4. Verificar el acceso por `https://easypanel.tudominio.com`.
5. Recién entonces, bloquear el acceso directo por el puerto 3000.

⚠ El orden es sagrado: **primero verificás que el dominio nuevo entra, después cerrás el 3000**. Si lo hacés al revés y algo sale mal con el certificado, te quedaste sin panel y sin forma de arreglarlo desde el panel.



- 📌 Este paso es también la primera demostración práctica de para qué servía el registro comodín: nadie cargó un registro `easypanel` en el panel DNS, y sin embargo `easypanel.tudominio.com` resuelve. Es exactamente el comportamiento descrito en la sección 1.12.1.

2.11. Endurecimiento mínimo

Las medidas siguientes constituyen la línea de base para cualquier servidor expuesto a internet. Ninguna es opcional en un entorno real.

#	Medida	Comando o ubicación
1	Autenticación solo por clave	PasswordAuthentication no en sshd_config
2	Firewall restrictivo	ufw default deny incoming
3	Verificación externa de puertos	nmap -Pn tudominio.com
4	Panel bajo HTTPS	Settings → Domain en Easypanel
5	Sistema actualizado	sudo apt update && sudo apt upgrade -y
6	Bloqueo de intentos repetidos	sudo apt install fail2ban
7	Actualizaciones automáticas de seguridad	sudo dpkg-reconfigure unattended-upgrades

fail2ban

Analiza los registros de acceso y bloquea temporalmente las direcciones que acumulan intentos fallidos. Con autenticación por clave el riesgo ya es bajo, pero reduce el ruido en los registros.

```
sudo apt install fail2ban -y
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
```

Actualizaciones automáticas

La enorme mayoría de los servidores comprometidos no cae por un ataque sofisticado sino por una vulnerabilidad **conocida, publicada y ya parcheada**. El intervalo entre que se publica una vulnerabilidad y que empieza a explotarse masivamente se mide en horas.

2.12. Principios de seguridad y evolución de las herramientas

En 1975, Jerome Saltzer y Michael Schroeder publicaron un artículo que enunciaba ocho principios de diseño para sistemas seguros. Medio siglo después siguen vigentes, y tres de ellos son literalmente el resumen de este capítulo.

Mínimo privilegio

Todo componente debe operar con los permisos mínimos necesarios para su función, y ni uno más. Aplicación directa: el contenedor de la base de datos no publica puertos porque no necesita ser alcanzable desde afuera.

Valores predeterminados seguros

Lo que no está explícitamente permitido, se niega. Es literalmente la primera línea de la configuración del firewall (default deny incoming), y es lo que Redis y MongoDB hacían al revés, con los resultados conocidos.

Economía del mecanismo

Cuanto más simple es la protección, más fácil es verificar que funciona. Por eso "no publicar el puerto" es preferible a "publicarlo y filtrarlo con reglas": no hay regla que auditar, nada que configurar mal. El puerto no existe.

❏ A esos tres se suma la **defensa en profundidad**: ninguna capa se asume infalible, y por eso se apilan varias independientes. En este servidor hay al menos cuatro —clave en lugar de contraseña, firewall restrictivo, no publicar puertos internos, fail2ban— y cada una supone que las otras pueden fallar.

Evolución de las herramientas: nftables y Ed25519

De iptables a nftables

Desde 2014 existe el reemplazo de iptables: **nftables**, que unifica en un solo marco lo que antes eran cuatro herramientas separadas (iptables, ip6tables, arptables, ebtables), usa una sintaxis más regular y evalúa las reglas de forma más eficiente.

Las distribuciones modernas —incluida Ubuntu 24.04— **ya usan nftables por debajo**: el comando iptables sigue existiendo pero es una capa de compatibilidad que traduce a nftables.

El comando actual para inspeccionar el estado real del sistema es:

```
sudo nft list ruleset
```

Muestra —entre muchas otras cosas— las cadenas que Docker registró.

Por qué Ed25519 y no RSA

La orden `ssh-keygen -t ed25519` elige un algoritmo de firma concreto, y la elección no es arbitraria. **Ed25519** es un esquema de firma sobre curvas elípticas, normado para SSH en la RFC 8709. Frente al RSA de 4096 bits ofrece tres ventajas:

- Claves mucho más cortas con seguridad equivalente o superior
- Firmas notablemente más rápidas
- **Muchas menos formas de configurarlo mal**: Ed25519 tiene un solo tamaño y no hay perilla que tocar

Es economía del mecanismo aplicada a la criptografía.

2.13. Verificación

Las siete comprobaciones siguientes cierran el capítulo. Las cuatro primeras validan el acceso; las tres últimas validan el cierre. Solo una de todas ellas constituye evidencia externa.

#	Comprobación	Cómo	Resultado esperado
1	Acceso por clave	ssh root@tudominio.com	Entra sin pedir contraseña
2	Contraseña deshabilitada	Intentar desde un equipo sin clave	Acceso denegado
3	Los cuatro integrantes acceden	Cada uno desde su equipo	Todos entran
4	Firewall activo	sudo ufw status verbose	active, con 22, 80 y 443
5	Puertos reales desde afuera	nmap -Pn tudominio.com	Solo 22, 80 y 443
6	Panel por HTTPS	https://easypanel.tudominio.com	Candado válido
7	Puerto 3000 inaccesible	http://IP_DEL_VPS:3000	No responde

⊗ La comprobación 5 es la única que vale como prueba de que el servidor está cerrado. Las demás describen intenciones; esa describe hechos.

2.14. Errores frecuentes

Síntoma	Causa	Resolución
Se pierde el acceso tras habilitar ufw	Se habilitó antes de permitir el 22	Consola de emergencia de Hostinger
Se pierde el acceso tras deshabilitar contraseñas	La clave pública no estaba bien instalada	Consola de emergencia; revisar <code>authorized_keys</code>
ufw dice cerrado y nmap dice abierto	Puerto publicado por Docker (sección 2.8.1)	Dejar de publicarlo; usar red interna
El certificado del panel no se emite	El registro comodín no resuelve, o el 80 está cerrado	Revisar DNS y ufw
Un integrante entra y otro no	Falta su clave pública en el servidor	Agregarla a <code>authorized_keys</code>
Permission denied (publickey)	Se usó la clave privada de otro equipo	Cada uno usa su propio par
Advertencia grande de que cambió la clave del anfitrión	El servidor se reinstaló (o hay suplantación)	Verificar el motivo antes de borrar la línea de <code>known_hosts</code>
nmap no devuelve nada	El proveedor descarta los pings	Agregar la bandera <code>-Pn</code>

2.15. Actividades

Actividad 1 — Inventario de puertos

Ejecutar `ss -tlnp` y confeccionar una tabla con cada puerto en escucha, el proceso responsable, la interfaz y una justificación de por qué debe o no estar accesible desde internet. Indicar a qué rango de la RFC 6335 pertenece cada uno.

Actividad 2 — Contraste entre lo declarado y lo real

Comparar la salida de `ufw status verbose` con la de `nmap -Pn` ejecutado desde otro equipo. Documentar toda discrepancia y explicar su causa en términos del recorrido del paquete.

Actividad 3 — Medición de la superficie de ataque

Contar los intentos de acceso fallidos acumulados desde la creación del servidor y estimar la tasa por hora. Identificar los diez países o rangos de origen más frecuentes.

Actividad 4 — Revocación de acceso

Eliminar la clave pública de un integrante de `authorized_keys`, verificar que pierde el acceso y que los demás lo conservan. Restaurarla después. Comparar el procedimiento con un esquema de contraseña compartida.

Actividad 5 — Investigación

Buscar en Shodan o Censys cuántas instancias de PostgreSQL están expuestas públicamente en Argentina. Discutir cuántas de esas exposiciones serían explicables por el problema de la sección 2.8.

Actividad 6 — Lectura del conjunto de reglas real

Ejecutar `sudo nft list ruleset` y localizar las cadenas que creó `ufw`, las que creó Docker, y en qué punto de enganche está registrada cada una. Explicar por qué una regla de `ufw` no puede bloquear un puerto publicado por Docker.

Actividad 7 — El túnel SSH (opcional)

Investigar la opción `-L` de `ssh` y explicar cómo permitiría conectarse a un servicio que escucha solo en `127.0.0.1` del servidor. Relacionar con la segunda estrategia de la sección 2.8.3.

2.16. Síntesis del capítulo

1 El VPS es una ilusión sostenida por un hipervisor

El aislamiento lo hace cumplir el procesador, los recursos se comparten, y el estado completo de la máquina es un archivo. De ahí salen el precio, la velocidad de reinstalación y el vecino ruidoso.

2 SSH autentica primero al servidor, después al cliente

La autenticación por clave es un desafío criptográfico: la clave privada nunca se transmite, y por eso un servidor comprometido no compromete el acceso a otros equipos.

3 El firewall no es ufw: es netfilter

Evalúa las reglas en puntos de enganche distintos. Docker se saltea ufw porque su tráfico va por FORWARD y ufw escribe en INPUT. La única verificación válida es escanear desde afuera.

4 El descubrimiento es automático, permanente y no selectivo

Nadie te ataca a vos; atacan a todos, todo el tiempo. Un socket se identifica por una tupla; un puerto abierto es un proceso concreto escuchando en una interfaz concreta.

5 Mínimo privilegio, predeterminados seguros y economía del mecanismo son de 1975

Explican cada decisión de este capítulo. El puerto más seguro es **el que nunca se publicó**. De ahí la red interna de la Clase 5.

Referencias normativas: RFC 4251–4254 (SSH), RFC 8709 (Ed25519), RFC 6335 (puertos), RFC 9293 (TCP). Bibliografía: Popek & Goldberg (CACM, 1974); Saltzer & Schroeder (*Proceedings of the IEEE*, 1975); Nemeth et al., *UNIX and Linux System Administration Handbook* (5.ª ed.); Kurose & Ross, *Computer Networking* (8.ª ed.).